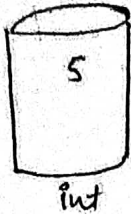


① What is Array?

- collection of similar type of elements.
- stored in contiguous memory location.
- linear data structure.



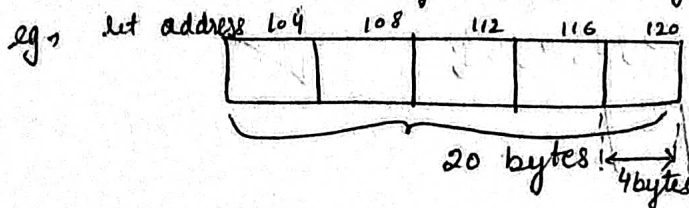
can we store 5 in this? Yes
"hello" → NO
5.5 → NO
true/false → NO

because it can only store integers.

② Behind the scene (memory)-

An integer takes 4 bytes of space.

so an array of 5 integers takes a total memory of $5 \times 4 = 20$ bytes (contiguous memory).



③ Why array is needed?

suppose we have to find greatest no. of two numbers. so we will make two variable, compare them and get the result.

Now suppose we want to find greatest element from 30/30000 elements. It's not convenient to create 30000 variables and compare.

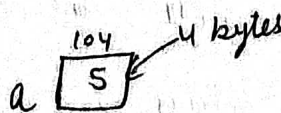
so that's why we use arrays to store element.

eg, `int array[30000];`

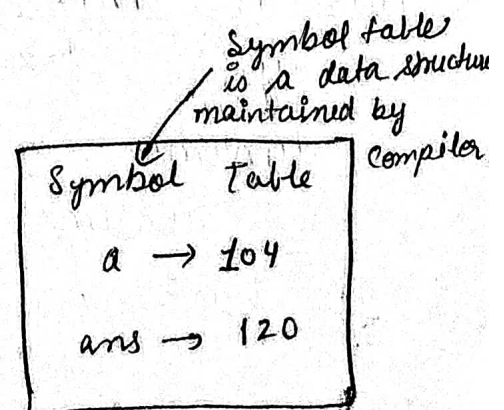
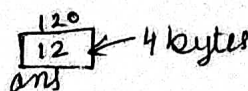
④ Creating an Array

creating variable `int a = 5;`

In memory →



`int ans = 12;`



→ declaration of array →

`int arr[10];`
↑ ↓ ↗
datatype name of size of the
the array array.

Contiguous space of 10 integers (40 bytes) is allocated.

• Address of an array -

`cout << arr;`
`cout << &arr;` } both gives address of the array.

Questions -

Create an array of

- int → 53 size → `int arr[53];`
- char → 50 size → `char arr[50];`
- bool → 23 size → `bool arr[23];`

→ Initialization -

`int arr[] = {1, 2, 3, 4, 5};` ✓
Yes, this size is not mandatory here.

⇒

1	2	3	4	5
---	---	---	---	---

`int arr[] = {1, 2, 3, 4, 5};` ✓

`int arr[10] = {1, 2, 3, 4, 5};` ✓

Now the question is what is stored in the rest of 5 spaces? ⇒ zero (0).

`int arr[4] = {1, 2, 3, 4, 5};` // Error.

Why Error?

Because the size of the array is 4 and we are storing 5 elements in it.

This is STATIC Array. Can we create an dynamic array? → fixed size.

dynamic array is creation of array at the runtime. (We will discuss it later).

We have one more way →

`int n;`
`cin >> n;`

`int arr[n];`
But this is BAD practice and not recommended.

• Questions-

• Create arrays

→ `int → 1, 2, 3, 5`

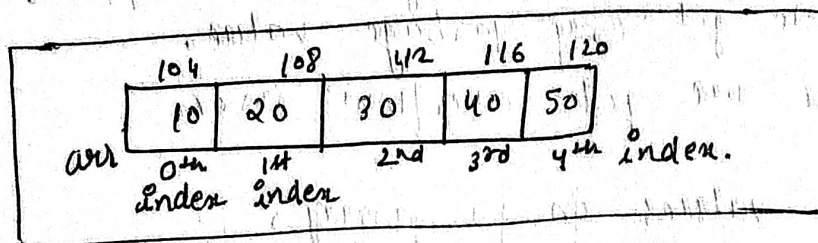
`int arr[4] = {1, 2, 3, 5};`

→ char array → a, b, c, d

`char array[4] = {'a', 'b', 'c', 'd'};`

⑥ Index and Access in array →

`int arr[5] = {10, 20, 30, 40, 50};`



5 size array → index → [0 - 4]

so n size array → index → [0 to (n-1)]

To access first element →

`arr[0] → 10`

same way we can access other elements with the help of indexes.

`arr[1] → 20, arr[2] → 30, arr[3] → 40, arr[4] → 50`

What is the significance of 0?

`arr[0]`

↙

`arr + 0 × 4`

`(arr + 0)`

↙
value at
this address

value at (104) address

= 10

`arr[1]`

value at `(arr + 1 × 4)`

value at `(104 + 1 × 4)`

value at `(104 + 4)`

value at 108 = 20

204	10	0
208	20	1
212	30	2
216	40	3
220	50	4

$arr[0] \rightarrow \text{value at } (arr + index)$
 $(\text{base address}) * \text{size of data type}$
 $= \text{value at } (200 + 0)$
 $= \text{value at } (200) = \underline{10}$

$arr[4] \rightarrow \text{value at } (arr + 4 \times 4)$
 $\rightarrow \text{value at } 216$
 $= \underline{40}$

$arr[10] \rightarrow \text{value at } (arr + 10 \times 4)$
 $= \text{value at } \underline{244}$

$\hookrightarrow$ (but this memory is not allocated for this array)

So in this case two things can happen.

- ① we get any garbage value.
- ② or we get overflow error.

① Printing all values in an array $\rightarrow$

```
int arr[] = {1, 3, 5, 7, 9};
```

```
for (int index = 0; index < 5; index++) {
    cout << arr[index];
}
```

// op $\rightarrow$ 1 3 5 7 9.

dry run -

```
for (i = 0; i < 5; i++)
{
    cout << arr[i];
}
```

$i = 0, 0 < 5 \rightarrow T$ $arr[0]$
print 1

$i = 1, 1 < 5 \rightarrow T$ $arr[1]$
print 3

$i = 2, 2 < 5 \rightarrow T$ $arr[2]$
print 5

$i = 3, 3 < 5 \rightarrow T$ $arr[3]$
print 7

$i = 4, 4 < 5 \rightarrow T$ $arr[4]$
print 9

$i = 5, 5 < 5 \rightarrow F$ ~~arr~~

② Taking array's element as an input $\rightarrow$

```
int arr[10];
for (int index = 0; index < 10; index++) {
    cin >> arr[index];
}
```

```
int n;
cin >> n;
arr[i] = n;
```

② instead of using `int arr[n]`;

we can use `int arr[10000];`

↑ a big number.

even if we are using ^{to a big number,} storing only 10 elements in it.

Q1 → Take 5 element i/p in array and print their doubles.

Sol.

```
int arr[5];  
for (int i = 0; i < 5; i++) {  
    cin >> arr[i];  
}  
for (int i = 0; i < 5; i++) {  
    { arr[i] *= 2;  
      cout << arr[i];  
    }  
}
```

// Taking 5 element in input.

// printing their doubles.

or simply remove these two lines with a single line → cout << 2 * arr[i];

Q2. We have an array -

$$\text{arr}[] = \{1, 3, 5, 7, 9\};$$

change it's all elements to 1 like this $\rightarrow [1, 1, 1, 1, 1]$

```
Sol-
int arr[] = {1, 3, 5, 7, 9};
for (int i = 0; i < 5; i++) {
    arr[i] = 1;
}
```

9
H.W → Find out what does memset func do and ~~the~~ how to use it.

```

① int arr[10] = {1, 2};
   for (int i = 0; i < 10; i++) {
       cout << arr[i];
   }

```

o/p $\rightarrow$ 1 2 0 0 0 0 0 0 0 remaining elements are 0's.

```

① int arr[10];
   for (int i = 0; i < 10; i++) {
       cout << arr[i];
   }

```

0/0 $\rightarrow$ all 10 garbage values.

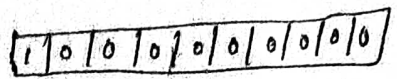
① To initialize all elements with 0 $\rightarrow$

$$\text{int arr}[10] = \{0\};$$
[illegible]

① How to initialize all elements with 1?

```
1 => int arr[10];
    for (int i=0; i<10; i++){
        arr[i] = 1;
    }
```

This will not work
arr[10] = {1};

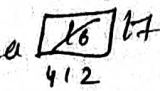
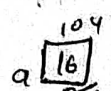


2 => memset, Explore it.

② Arrays & functions ->

```
main() {
    int a = 16;
    inc(a);
    cout << a;
}
```

pass by value.
copy
inc(int a) {
 a++;
 cout << a;
}



16 is printed.

17 is printed.

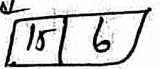
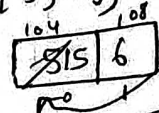
In array ->

```
main() {
    int arr = {5, 6};
    inc(arr);
    printarray(arr);
}
```

```
inc(int arr[]) {
    arr[0] = arr[0] + 10;
    printarray(arr);
}
```

Name can be different.
but the location is same.

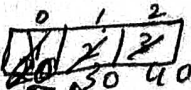
suppose printarray is a function which prints all element of the array.



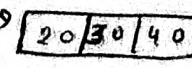
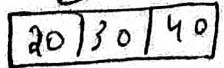
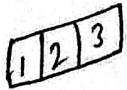
so this is pass by reference.

Whenever an array is passed in a function it is always passed by reference.

```
main() {
    int a = {1, 2, 3};
    print(a);
    fun(a, size);
    print(a);
}
```



```
fun(a[], size) {
    a[0] = 20;
    a[1] = 30;
    a[2] = 40;
    print(a);
}
```



- sizeof → gives the size of an array.
But what happens when only some elements are there in array (not all).

`int arr[10] = {1, 2};`

In this case `sizeof` operator does not give us the value ~~==~~ filled with element but it gives the size allocated. (In this case 40).

⑥ Linear Search in Array

arr

2	9	6	7	4	12	15
---	---	---	---	---	----	----

→ 6 is present in this array or not?
⇒ Yes, it's present.

How do we know?

⇒ Go to each element and compare.

`arr[i] == 6` ?

Let's code this -

```
main() {
    int arr[5] = {1, 3, 5, 7, 8};
    int size = 5;
    cout << "Enter the key to find";
    int key;
    cin >> key;
    if (linearSearch(arr, size, key)) {
        cout << "Found" << endl;
    }
    else {
        cout << "Not Found" << endl;
    }
}
```

```
bool linearSearch (int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key)
            return true;
    }
    return false;
}
```

• Alternate way - (w/o function)

```
main() {
```

```
    int arr[] = {1, 2, 3, 4, 5};
```

```
    int size = 5;
```

```
    int key = 3;
```

```
    bool flag = 0;
```

```
    for (int i = 0; i < size; i++) {
```

```
        if (arr[i] == key) {
```

```
            flag = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (flag)
```

```
        cout << "Present" << endl;
```

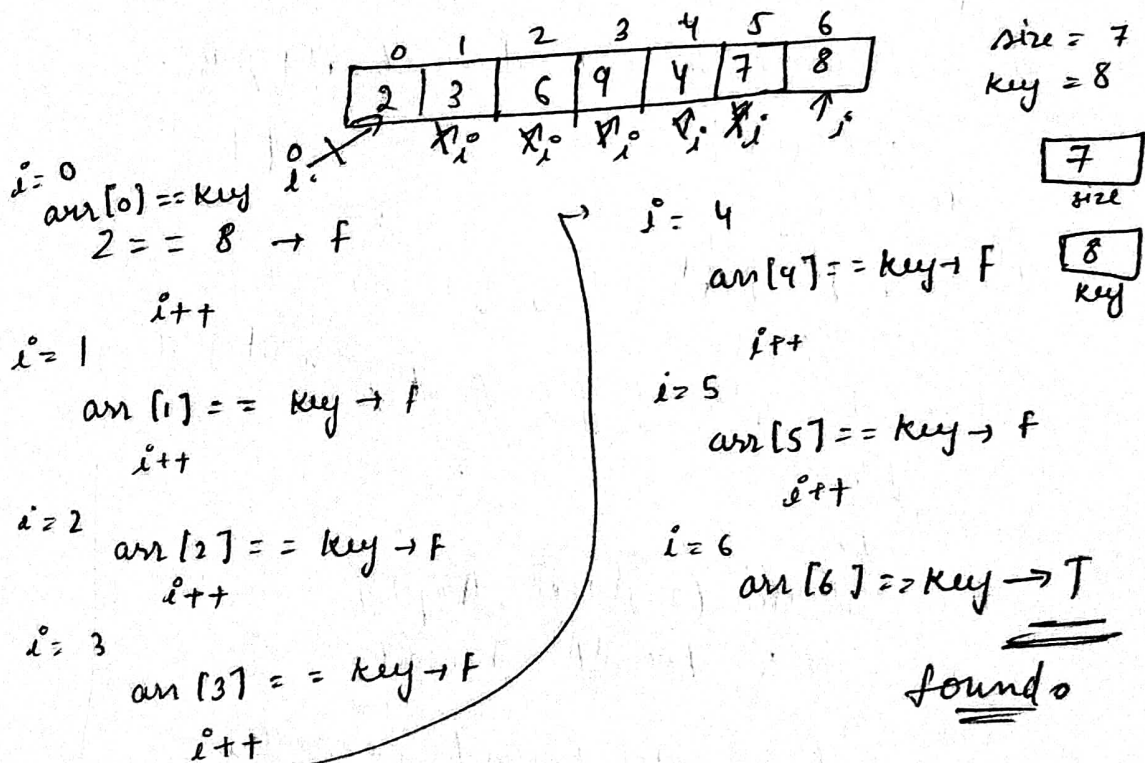
```
    else
```

```
        cout << "Not present" << endl;
```

```
}
```

flag → 1 (found the element)

flag → 0 (not found)



① Count 0's and 1's in an array.

```
int arr[] = {0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0};
```

```
int size = 13;
```

```
int numZero = 0;
```

```
int numOne = 0;
```

```
for (int i = 0; i < size; i++) {
```

```

    if (arr[i] == 0)
        numZero++;
    if (arr[i] == 1)
        numOne++;
}
}

```

We could have used else here but in case of array contains other than 0 and 1 than using else is a problem.

cout << "Number of Zeros : " << numZero << endl;

cout << "Number of Ones : " << numOne << endl;

① Count no. of 2's →

1	2	3	2	4	6	2	9	9	4	1	2
---	---	---	---	---	---	---	---	---	---	---	---

int cnt = 0;

```

for (int i = 0; i < size; i++) {
    if (arr[i] == 2)
        cnt++;
}

```

cout << "No of 2's : " << cnt << endl;

② Maximum no. in an array →

0	1	2	3	4	5	6
2	4	1	6	8	9	0

~~maxNum~~ = ~~INT_MIN~~ ~~2 4 6 8 9~~

Include
a header
file,
limits.h

```

int size = 7;
int maxi = INT_MIN;
for (int i = 0; i < size; i++) {
    if (arr[i] > maxi)
        maxi = arr[i];
}

```

cout << "Maximum no. is " << maxi << endl;

Best practice (when you have to find max no., use INT_MIN to initialize and when you have to find min, use INT_MAX as initialization.)

③ Minimum no. in an array →

```

int arr[] = {2, 4, 6, 1, 3, 7, 9, 12, 56, 43, 21};
int size = 11;
int mini = INT_MAX;

```

```

for (int i = 0; i < size; i++) {
    if (arr[i] < mini)
        mini = arr[i];
}

```

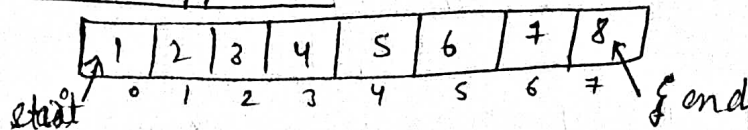
cout << "Minimum no. is : " << mini << endl;

① Extreme print in Array-

i/p $\rightarrow \{1, 2, 3, 4, 5, 6, 7, 8\}$

o/p $\rightarrow 1, 8, 2, 7, 3, 6, 4, 5$

Two pointer's approach-



start $\rightarrow$ starting index $\rightarrow 0$

end $\rightarrow$ last index $\rightarrow (\text{size}-1)$

loop.

- $\rightarrow$ print arr[start]
- print arr[end]
- start ++ $\leftarrow$ increment start
- end -- $\leftarrow$ decrement end.

Rukna kab hai?

$\hookrightarrow$ jaise hi start end se aage nikal jayega.

```
int arr[] = {10, 20, 30, 40, 50, 60, 70, 80};
```

```
int size = 8;
```

```
int start = 0, end = size - 1;
```

```
while (start < end)
```

```
while (true) {
```

```
    if (start > end)
```

```
        break;
```

```
    cout << arr[start] << " ";
```

```
    cout << arr[end] << " ";
```

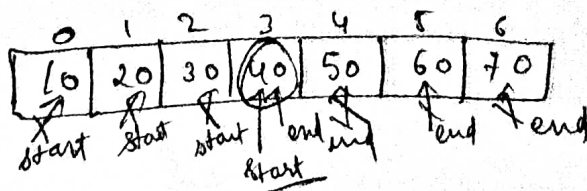
```
    start++;
```

```
    end--;
```

```
}
```

o/p $\rightarrow 10\ 80\ 20\ 70\ 30\ 60\ 40\ 50$

In case of odd element one element will be printed 2 times.



o/p $\rightarrow$ 10 20 30 40 50 40 30

because start and end pointing at the same element.

~~so with~~ so we will just add a simple condition.

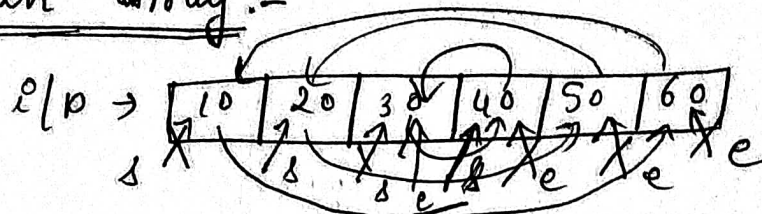
if (start == end)
 $\text{cout} \ll \text{arr}[\text{start}] \ll " ";$

or here we can also write
 $\text{arr}[\text{end}]$.
 (either of them).

Now while condition

$\Rightarrow$ while (start <= end)

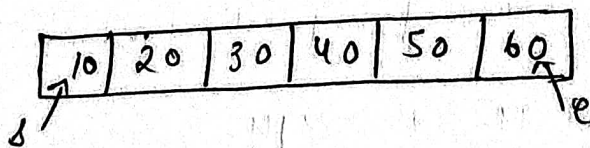
⑥ Reverse an array:-



o/p $\rightarrow$ [60 | 50 | 40 | 30 | 20 | 10]

$\Rightarrow$ we only swapped extreme points.

we have a predefined function to swap.



s = 0
 e = n - 1

let's code this $\rightarrow$

int arr[] = { 10, 20, 30, 40, 50, 60 };

int size = 6;

int start = 0, end = size - 1;

while (start <= end) {

swap(arr[start], arr[end]);

start ++;

end --;

}

Steps $\rightarrow$

- ① swap(arr[s], arr[e])
- ② s++;
- ③ e--;